07 搜索与索引

- 搜索与索引
- 基于顺序表的搜索
- 二叉搜索树

作业

- 设二叉树用二叉链表表示，树中结点的关键码互不相同，试编写算法判别给定的二叉树是否为二叉搜索树
- **空二叉树T是一棵二叉搜索树**
- **若T非空，首先是它的左、右两棵子树都是二叉搜索树，还要求左子树中的所有结点的键值都小于根结点的键值，并且右子树中的所有结点的键值都大于根结点的键值**
- 为了能简单实现子树所有结点与根结点比较，对于左子树可用左子树中最大的键值，对于右子树可以用右子树中最小的键值，即若左子树中的最大键值比根结点键值小，并且右子树中的最小键值比根结点键值大，就能得出这是一棵二叉搜索树的结论

作业

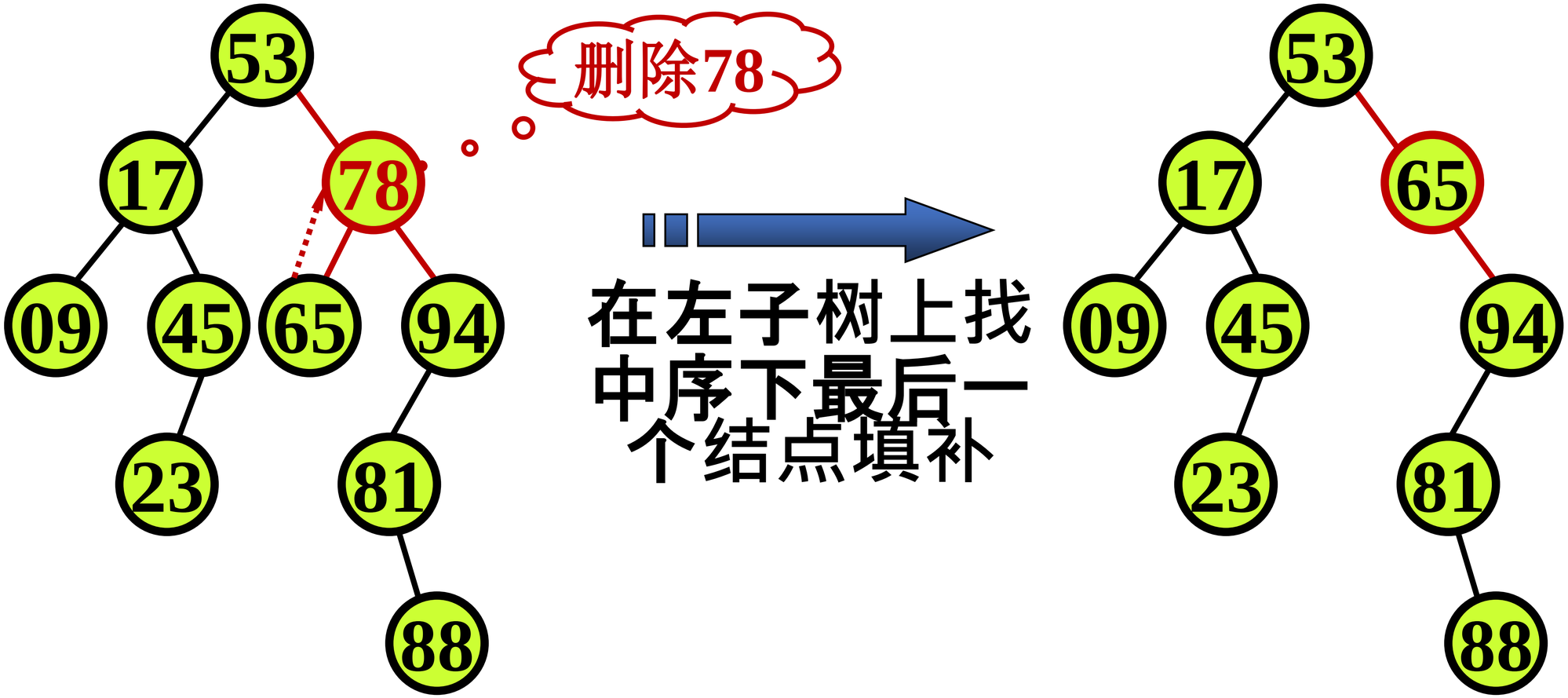
- 设二叉树用二叉链表表示，树中结点的关键码互不相同，试编写算法判别给定的二叉树是否为二叉搜索树
- **二叉搜索树的中序遍历可得到键值从小到大的结点序列**
- **可以在中序遍历中设置一个指向当前访问结点的中序直接前驱的指针pre，每访问一个结点，就比较当前访问结点的键值是否大于pre所指结点的键值，只要有一个结点不满足，则此树就不是二叉搜索树**

作业

- 设二叉搜索树用二叉链表表示，树中结点的关键码为整数，试编写算法将关键码小于给定值x的结点全部删除
- **若根结点rt->data <= x，则rt指向rt的右子女，删除rt及rt的左子树的全部结点，重复这一步，直到rt->data > x**
- **沿rt的左分支搜索，直到左分支结点的data <= x，回到上步继续执行**

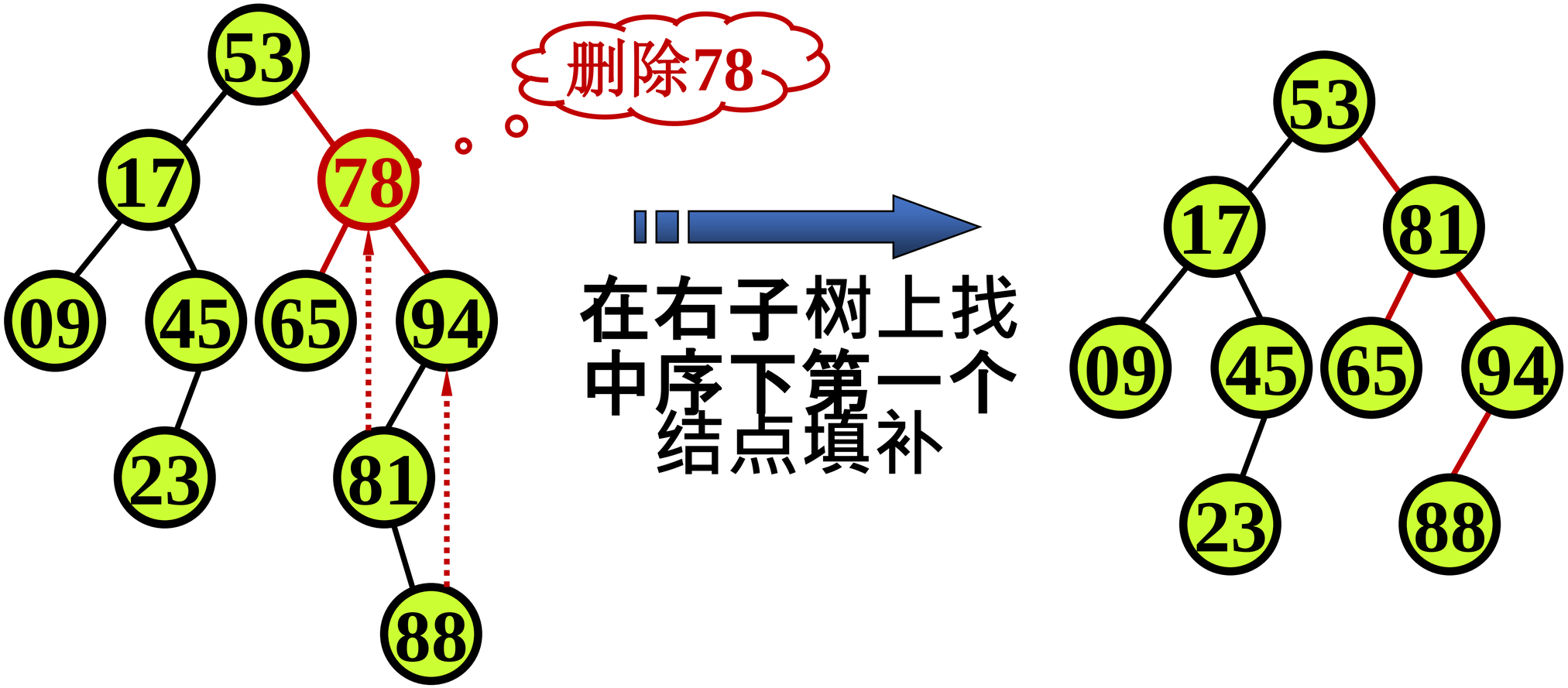
作业

- 在二叉搜索树上删除一个有两个子女的结点时，可以采用以下方法：用左子树上具有最大关键码的结点或者右子树上具有最小关键码的结点顶替，再递归地删除适当的结点，试编写算法实现这个删除方法



作业

- 在二叉搜索树上删除一个有两个子女的结点时，可以采用以下方法：用左子树上具有最大关键码的结点或者右子树上具有最小关键码的结点顶替，再递归地删除适当的结点，试编写算法实现这个删除方法

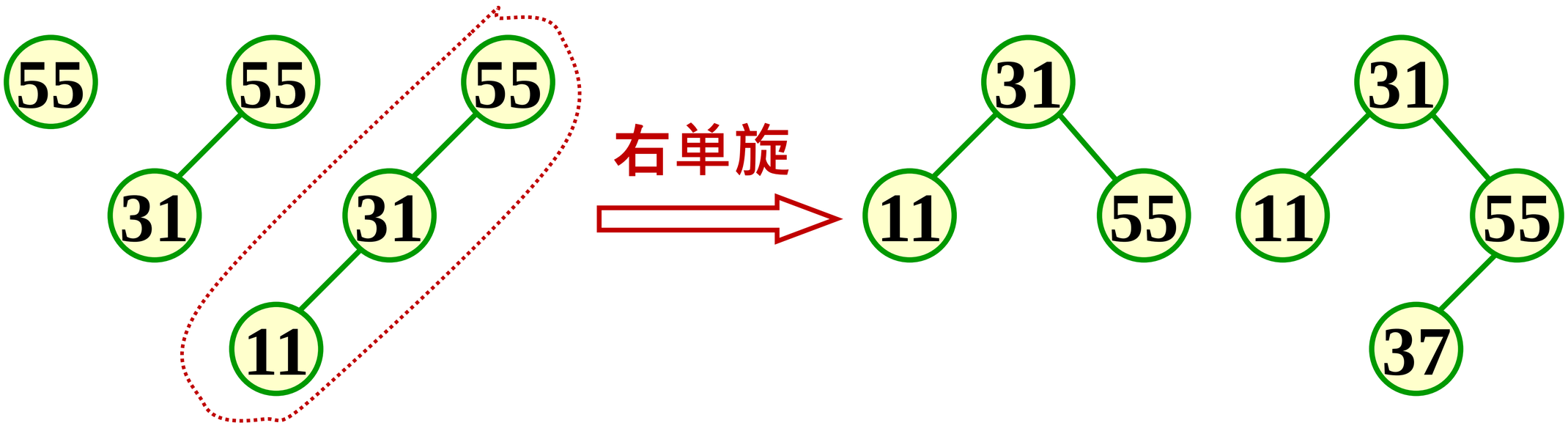


07 搜索与索引

- AVL树
- Hash搜索

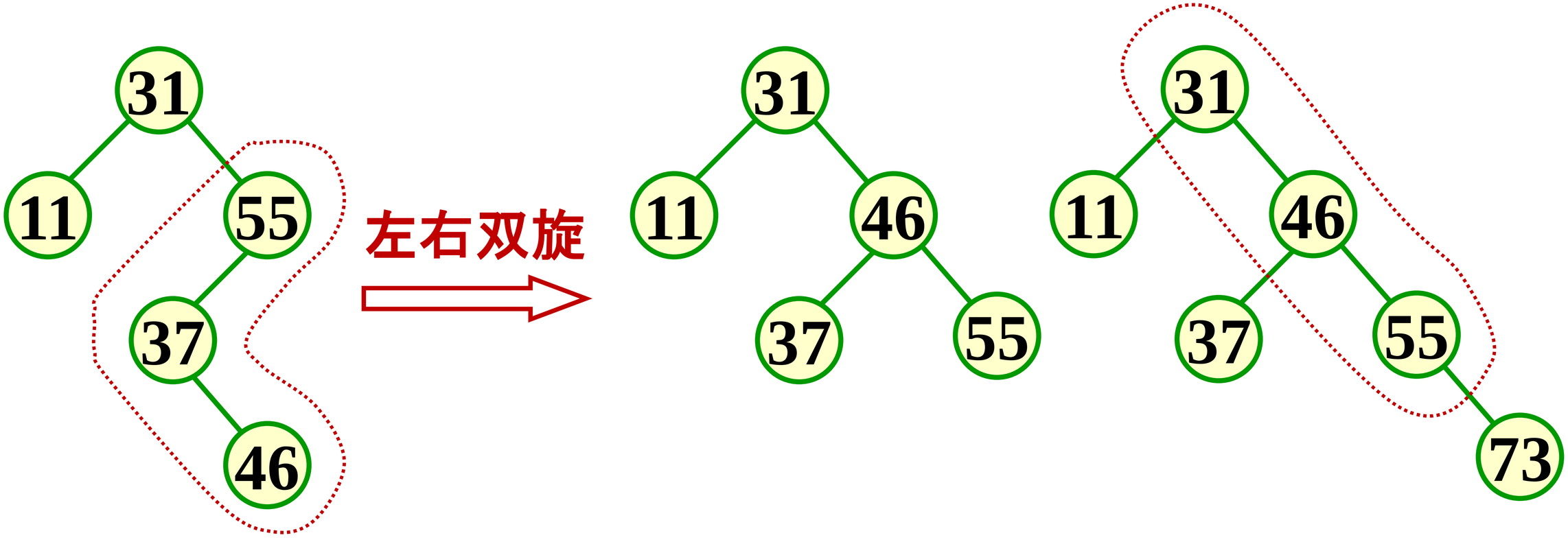
作业

- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度



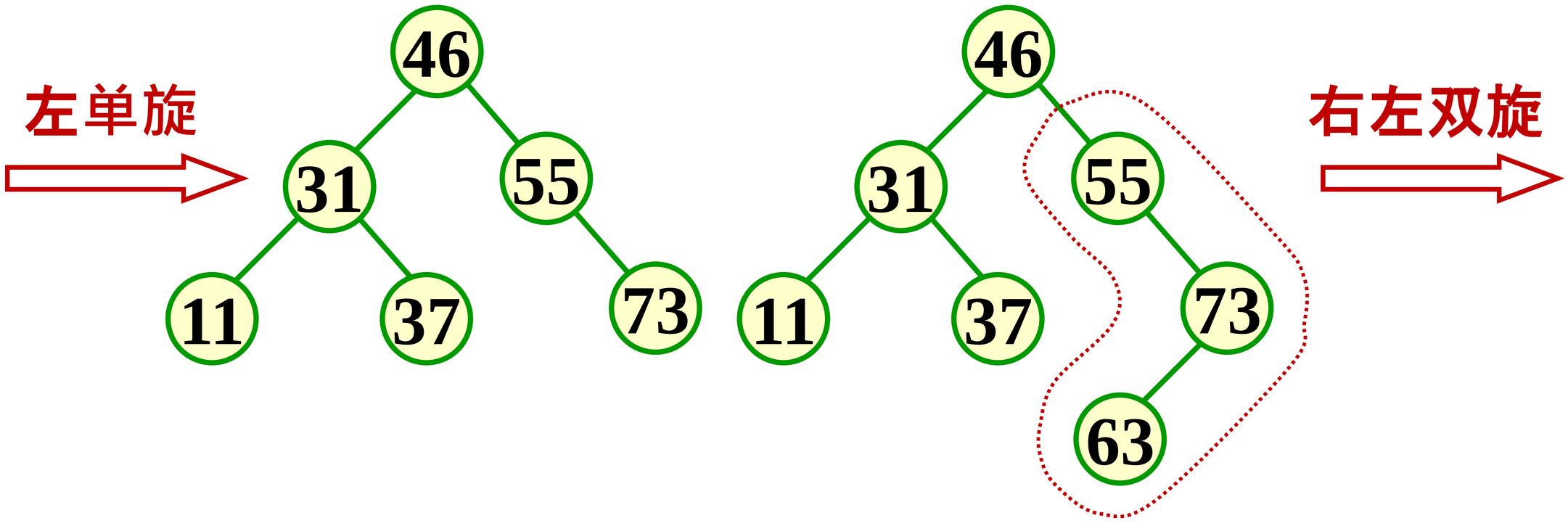
作业

- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度



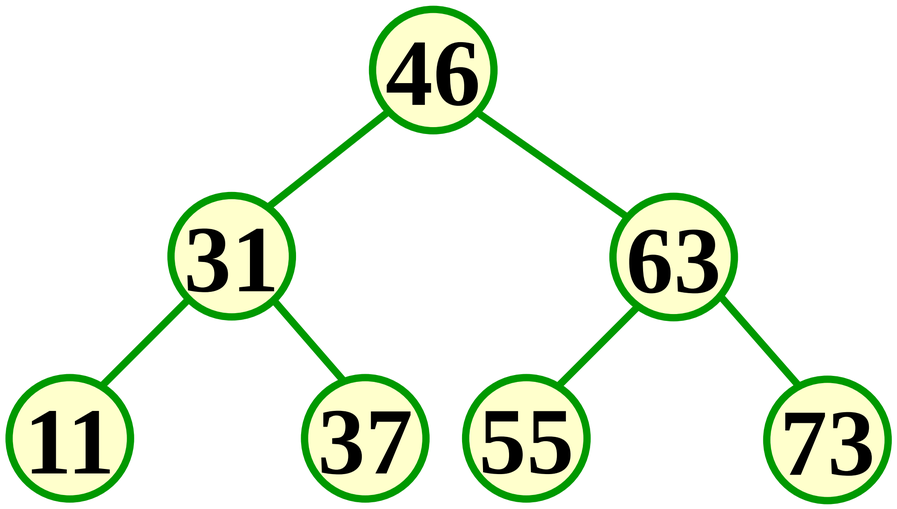
作业

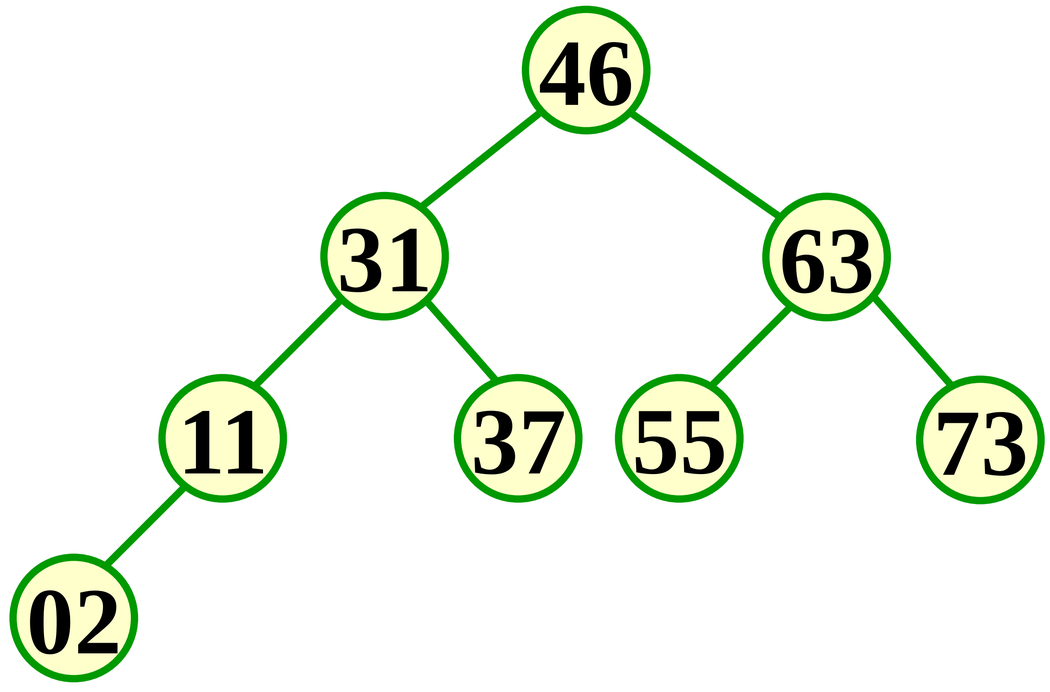
- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度



作业

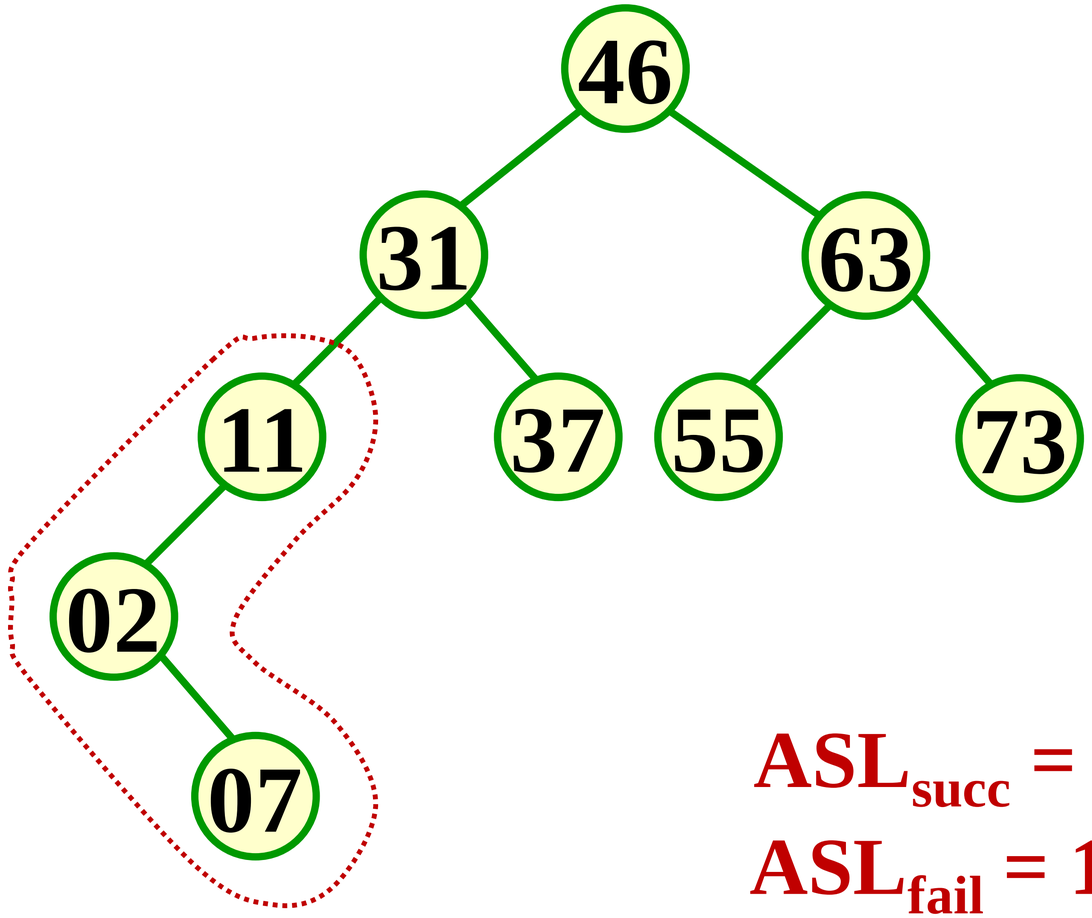
- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度



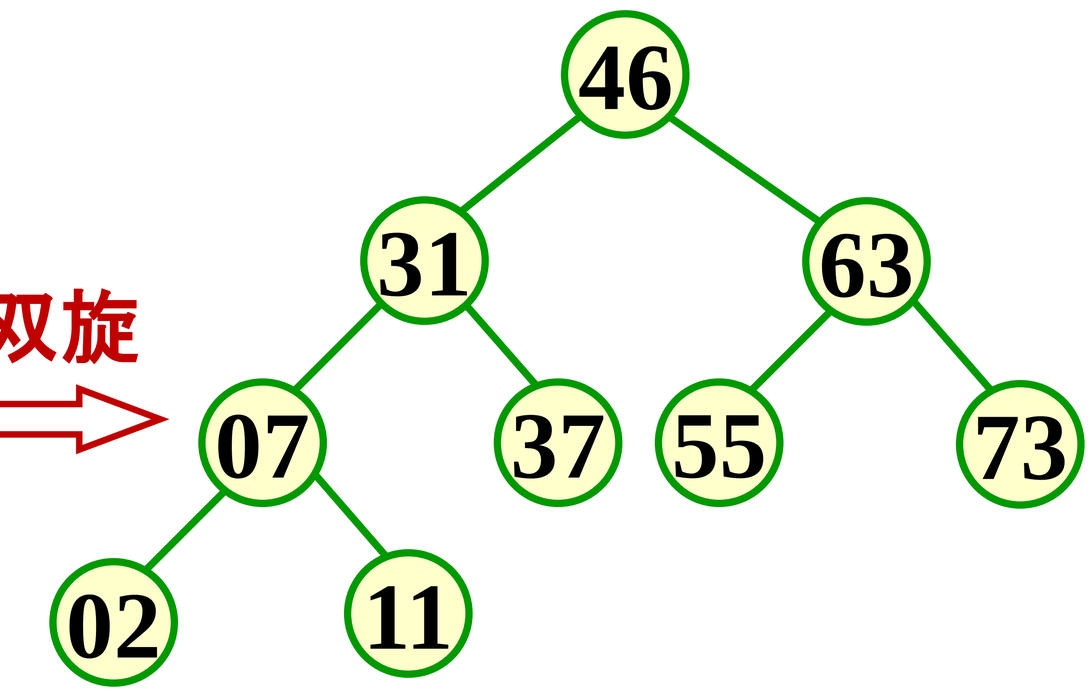


作业

- 有一个关键码的输入序列{55, 31, 11, 37, 46, 73, 63, 02, 07}
  - 从空树开始构造AVL树，画出每插入一个结点时的二叉树，若发生不平衡，指明需要的平衡旋转类型以及平衡化后的AVL树
  - 计算最终的AVL树在等概率情况下搜索成功/失败的平均搜索长度



左右双旋



$ASL_{succ} = 1/9*(1*1+2*2+4*3+2*4) = 25/9$

$ASL_{fail} = 1/10*(4*4+6*3) = 34/10$

作业

- 设Hash表为HT[13]，Hash函数为Hash(key) = key%13，用闭散法解决冲突，对关键码12, 23, 45, 57, 20, 03, 78, 31, 15, 36依次插入Hash表
  - 计算采用线性探查法时的Hash表，计算等概率下搜索成功/失败的平均搜索长度

0	1	2	3	4	5	6	7	8	9	10	11	12
78		15	03		57	45	20	31		23	36	12
(1)		(1)	(1)		(1)	(1)	(1)	(4)		(1)	(2)	(1)

$ASL_{succ} = 1/10*(8*1+1*2+1*4) = 14/10$

$ASL_{fail} = 1/13*(2+1+3+2+1+5+4+3+2+1+5+4+3) = 36/13$

作业

- 设Hash表为HT[13]，Hash函数为Hash(key) = key%13，用闭散法解决冲突，对关键码12, 23, 45, 57, 20, 03, 78, 31, 15, 36依次插入Hash表
  - 计算采用双散列法时的Hash表，ReHash(key) = (7*key)%10+1，计算等概率下搜索成功的平均搜索长度

0	1	2	3	4	5	6	7	8	9	10	11	12
78		15	03		57	45	20	31	36	23		12
(1)		(1)	(1)		(1)	(1)	(1)	(3)	(5)	(1)		(1)

$$ASL_{succ} = 1/10*(8*1+1*3+1*5) = 16/10$$

作业

- 有150个记录要存储到Hash表中，并用线性探查法解决冲突，要求找到所需记录的平均比较次数不超过2次，那Hash表需要多大？设a是Hash表的装填因子，ASL_{succ} = (1+1/(1-a))/2
- (1+1/(1-a))/2 <= 2
- a <= 2/3
- n/m = 150/m <= 2/3
- m >= 225

08 图

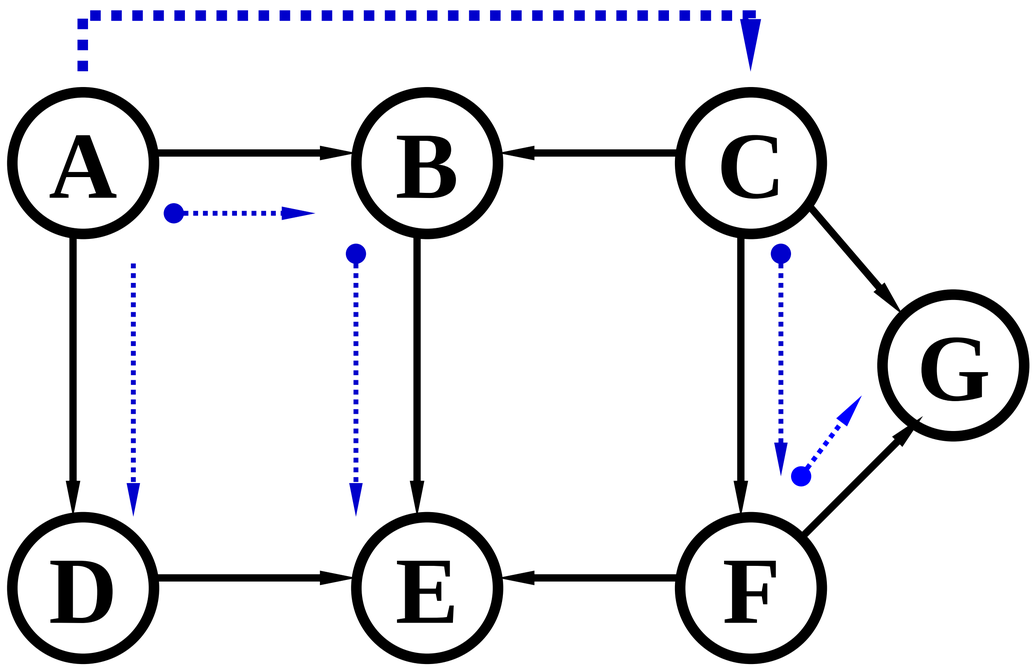
- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

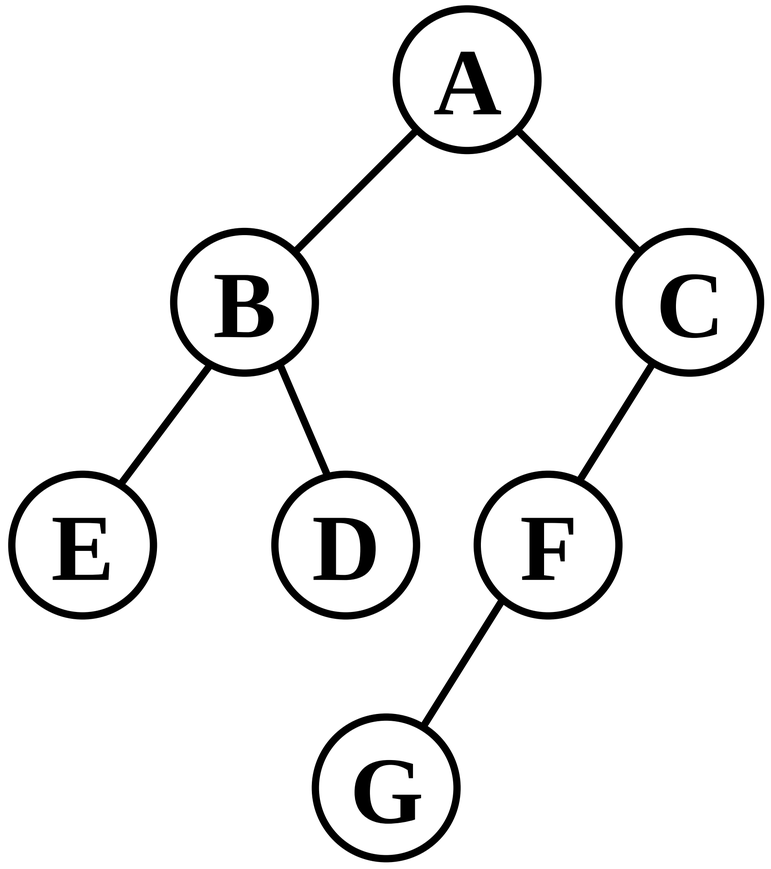
作业

- G是有n个顶点的连通图，证明G至少有n-1条边
- 可用数学归纳法证明
- 当n=2时，图G连通只需要一条边，结论成立
- 设n=k (k>2)时结论成立
- 则当n=k+1时，即增加一个顶点p后，若p与图中原有的k个顶点没有边，则显然p与图不连通；若要求k+1个顶点做成的图是连通图，p必须与图中原有的k个顶点中的任何一个顶点有边相连，即k条边就可使图连通，结论成立

作业

- 改造深度优先搜索算法，在遍历图的过程中建立生成森林的左子女-右兄弟链表





作业

- 改造深度优先搜索算法，在遍历图的过程中建立生成森林的左子女-右兄弟链表

```
template <class NameType, class DistType>
void DFS_Forest(Graph <NameType, Disttype> &G, Tree <NameType> &T) {
    TreeNode <NameType> *p, *q;  int i;  int n = G.VerticesNumber( );
    int *visited = new int[n];  // 访问数组
    for (i = 0; i < n; i++) visited[i] = 0;
    for (i = 0; i < n; i++)  // 遍历所有顶点
        if (!visited[i]) {  // 顶点i未访问过
            p = new TreeNode<NameType>();  p->data = G.GetValue(i);
            p->firstChild = p->nextSibling = null;
            if (T.IsEmpty( )) T.root = p;  // 顶点i成为根
            else q->nextSibling = p;  //顶点i成为subT右兄弟
            q = p;
            DFS_Tree(G, p, i, visited);  // 从顶点i出发深度优先遍历，建子树
        }
}
```

作业

- 改造深度优先搜索算法，在遍历图的过程中建立生成森林的左子女-右兄弟链表

```
template <class NameType, class DistType> void DFS_Tree(Graph <NameType, DistType> &G, TreeNode <NameType> *rt, int v,  int visited [ ]) {
    TreeNode <NameType> *p, *q;  visited[v] = 1;  int w = G.GetFirstNeighbor(v);
    int firstChild = 1;  // 第一个未访问子女应是v的左子女
    while (w != -1) {  // 邻接顶点w存在
        if (!visited[w]) {  // w未访问过，将成为v的子女
            p = new TreeNode<NameType>();  p->data = G.GetValue(w);
            p->firstChild = p->nextSibling = null;
            if (firstChild) { rt->firstChild = p;  firstChild = 0; }  // p插入为rt的左子女
            else q->nextSibling = p;  // p插入为q的右兄弟
            q = p;  // q指向兄弟链最后结点
            DFS_Tree(G, q, w, visited);  // 递归建立以q为根的子树
        }
        w = G.GetNextNeighbor(v, w);  // 取v的下一个邻接顶点w
    }
}
```

作业

- 给定一个连通图G，所有边都没有权值，编写一个算法求从顶点v能到达的最短路径为k的所有顶点(最短路径长度以边数为单位计算)
- 使用图的广度优先遍历的层次特性来求解，也即要在以v为起点调用BFS算法输出第k+1层上的所有顶点
- 在访问顶点时需要知道层数，而每个顶点的层数是由其前驱决定的(起点除外)，因此可以从第一个顶点开始，每访问到一个顶点就根据其前驱的层数计算该顶点的层数，并将层数与顶点编号一起入队、出队(实际上可增加一个队列来保存顶点的层数，并且将层数的相关操作与对应顶点的操作保持同步，即一起置空、出队和入队)

08 图

- 最短路径
- 拓扑排序
- 关键路径

作业

- 为判断图中是否存在环(回路)给出尽量多的方法，说明每一个方法的具体思路
- 利用拓扑排序可以判定图G是否存在回路，即在拓扑排序输出结束后所余下的顶点均有前驱，则说明只得到了部分顶点的拓扑有序序列
- 设G是n个顶点的无向连通图，若G的边数>=n，则G中一定有回路存在，因此只要计算出G的边数，就可判定图G中是否存在回路

作业

- 为判断图中是否存在环(回路)给出尽量多的方法，说明每一个方法的具体思路
- 设G是n个顶点的无向连通图，若G的每个顶点的度大于或等于2，则图中一定存在回路
- 利用深度优先遍历算法可以判定图G中是否存在回路，即若深度优先遍历过程中遇到了回边(即之前访问到过的顶点)，则必定存在环

作业

- 若表示一个有向图的邻接矩阵中对角线以下的元素均为0，则该图的拓扑有序序列一定存在吗？为什么？
- 一定存在。对于上三角部分任意的i和j，A[i][j] = 1，则一定存在一条有向边<v_i, v_j>且i < j，表明所有的有向边都是顶点号小的指向顶点号大的，因此不存在回路，其拓扑有序序列一定存在

09 排序

- 基本排序方法
- 归并排序
- 快速排序
- 堆排序
- 希尔排序
- 基数排序

作业

- 设有n个元素随机存放于数组中，试编写程序返回数组中按关键码从小到大排序的第k个元素的值
- 可以采用快速排序的划分算法，理想情况下，每次减半，求解时间为$n+n/2+n/2^2+\ldots <= 2n-1$，即时间复杂度为O(n)
- 若初始排序已经接近有序，则直接采用插入排序

作业

- 设待排序的1000个关键码互不相同，其取值为1～10000的正整数，试编写程序以尽可能少的时间代价完成排序
- **桶排序**

Q&A

陈碧　欢

bhchen@fudan.edu.cn

https://chenbihuan.github.io